Stack - Stack is nothing but a data structure that holds a certain kind of data.
Last In First Out
Implemented using Arrays
Implemented using LinkedList
Implemented using Queue / Implemented using Stack

- push(els) - Adds an element to the top of the Stack.
- pop(els) - Removes and returns the top element of the Stack
- peek(els) - Returns the top element of the Stack, without removing it.
- size() - Returns the number of elements currently in the Stack.

Queue -

- First In First Out

- add(element) or offer(element) - Adds an element to the end of the queue. offer is generally preferred as it returns false if the queue has a capacity limit is full, while add throws an exception.

- remove() or poll() - Removes and returns the element at the front of the queue. poll returns

null if the queue is empty, whereas remove throws an exception.

- peek() or element() - Returns the element at the front of the queue without removing it. peek returns null if the queue is empty, while element throws an exception.

- size() - Returns the number of elements in the queue.

19:40 ⇒

PAGE NO.
DATE: / /

⇒ Implementing Stack using Array

```
import java.util.*;
public class StackImpl {
    public static void main(String args[]) {
        stack s = new stack();
        s.push(6);
        s.push(3)
        s.push(7);
        Syso(s.peek());
        Syso(s.size());
        Syso(s.pop());
        Syso(s.size());
        Syso(s.peek());
    }
}

class stack {
    int size = 10000;
    int arr[] = new int[size];
    int top = -1;
    void push(int x) {
        top++;
        if (top >= 10) Syso("exceeded");
        arr[top] = x;
    }
    int pop() {
        if (top == -1) ...
        int x = arr[top];
        top--;
        return x;
    }
```

When you have more elements than the size of the stacks

PAGE NO.
DATE: / /

```
int peek() {
    return arr[top];
}
```

if (top == -1) ...

```
int size() {
    return top+1;
}
```

```
}
```

output :-

7
3
7
2
3

Time Complexity –

push() – O(1)
pop() – O(1)
peek() – O(1)
size() – O(1)

Space Complexity –

O (size of arr)

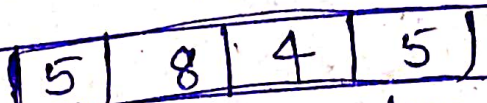
~~Time Complexity – O(n)~~

~~Space Complexity – O(n)~~

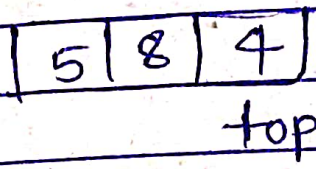
PAGE NO.
DATE:

DRY RUN OF STACK :-

push (5) push (8) push (4) push (5)

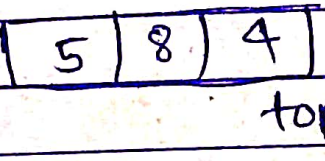


pop()



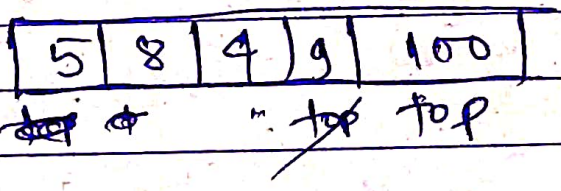
popu ⇒ 5

peek()

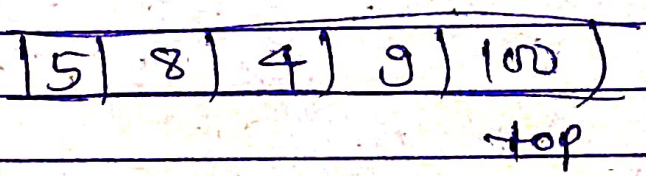


arr[top] = 4 ✓

push (9) push (100)

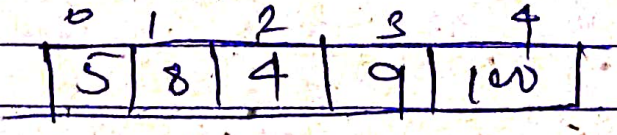


peek()



arr[top] = 100

size()



top+1
⇒ 5 ✓

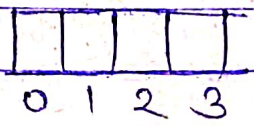
DATE: / /

DRY - RUN OF QUEUE

Step 01 :- ① size = 4
int Q [4]

② currSize = 0

push(3)
push(2)
push(4)

③ 

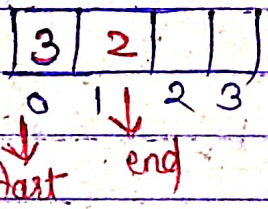
start == end == -1

Step 02 :- (push(1))
[currSize = 0 size = 4]

push(3)

currSize = 1

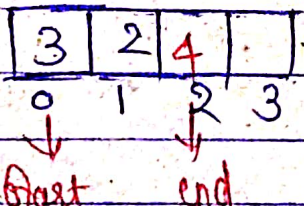
Step 03 :- [currSize = 1 size = 4]
(push(1))



push(2)

currSize = 2

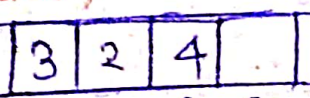
Step 04 :- [currSize = 2 size = 4]
(push(1))



push(4)

currSize = 3

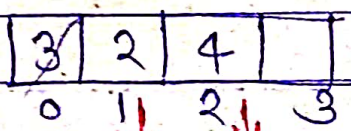
Step 05 :- (peek())



top ?? ⇒ arr[start]
(given that queue is not null)

Step 06 :- (pop())

[currSize = 3] ✓



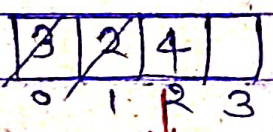
(pop())

start = start + 1
currSize == currSize - 1

currSize = 2

Step 07 :- (pop())

[currSize = 2] ✓



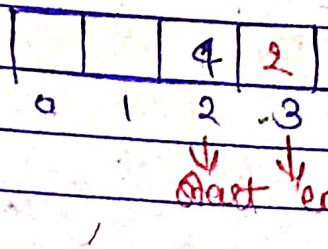
(popU)

start = start + 1
currSize -= 1;

currSize = 1

PAGE NO.
DATE: / /

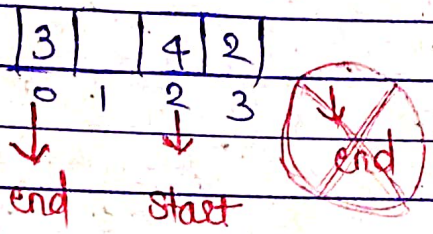
Step 08 :- (push()) (curSize = 1) ✓



push(2)

curSize = 2

Step 09 :- (push()) (curSize = 2) ✓

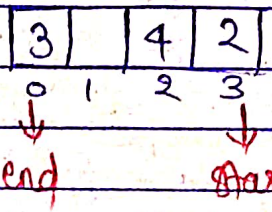


push(3)

end = (end+1) % size

curSize = 3

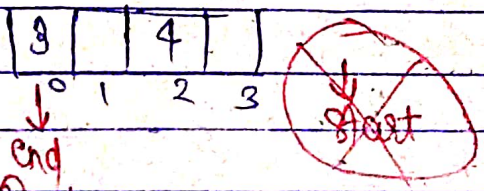
Step 10 :- (pop())



pop()

curSize = 2

Step 11 :- (pop())



curSize = 1

PAGE NO
DATE: / /

Step12 :- pop() (currSize = 1) ~ Destroying the queue

wherever the start & end would have been bring them to the -1

start == end == -1

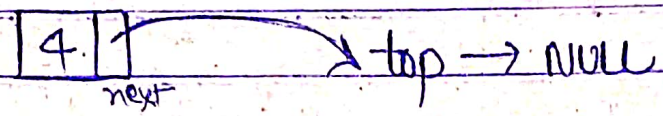
{ A lot of Edge cases }

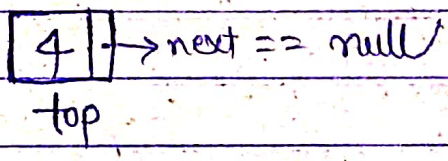
DRY RUN OF STACK (using LinkedList) :-

The Disadvantage of implementing Stack & Queue using Array is that Array has fixed size.

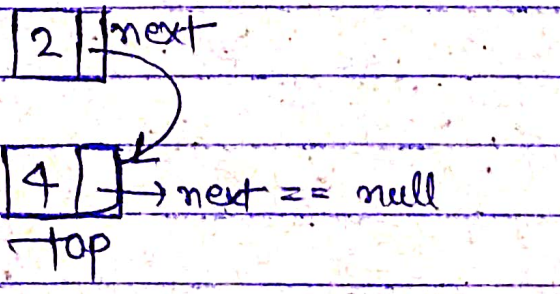
So Now for that (for dynamic size) we will use LinkedList...

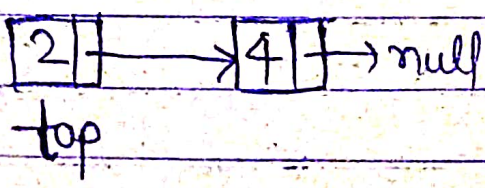
Step 01 :- push(4)



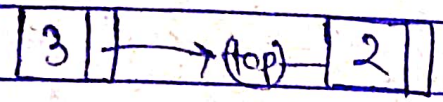


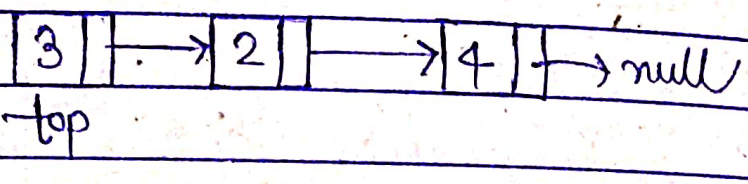
Step 02 :- push(2)



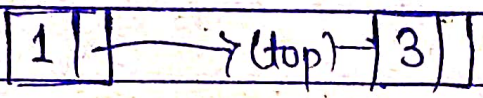


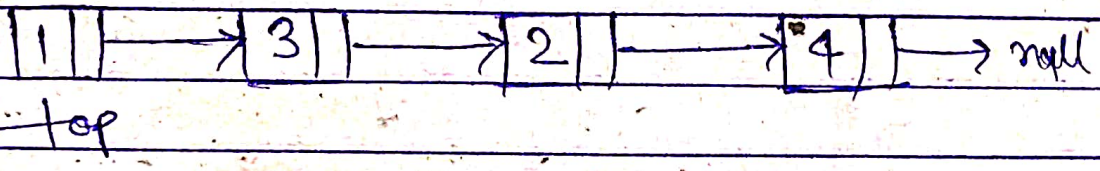
Step 03:- push(3)





Step 04:- push(1)

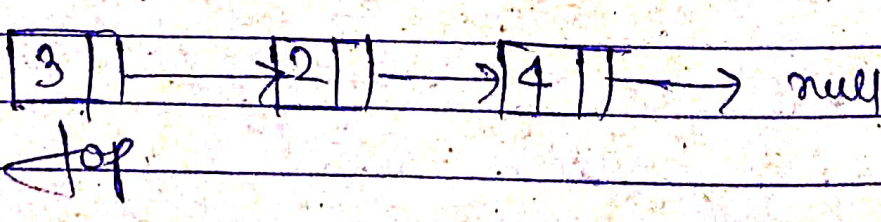




Step 05:- peek() ⇒ top.val (value)...

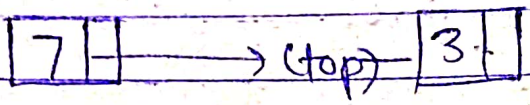
Step 06:- pop()

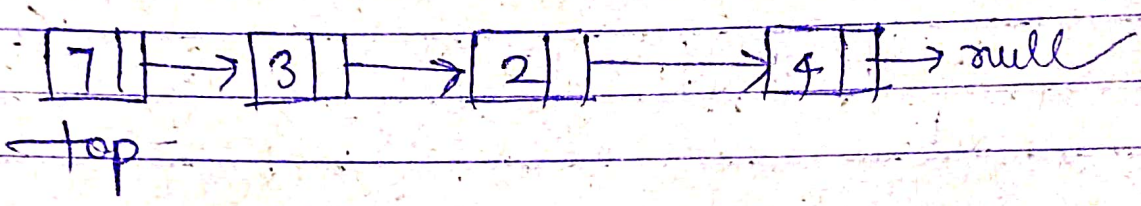
temp = top → in C++
top = top.next
delete(temp) → in C++



DATE: / /

Step 07:- push(7)





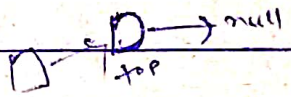
Step 08:- size

Count the number of push operations & Subtract when popping out.

PAGE NO.
DATE: / /

Implementing Stack using Linked List :-

```
class Stack {
  private class StackNode {
      int val;
      StackNode next;
      StackNode (int v) {
          val = v;
          next = null;
      }
  }
  StackNode top;
  int size;
  Stack () {
      this.top = null;
      this.size = 0;
  }
```




```
  public void StackPush (int x) {
      StackNode element = new StackNode(x);
      element.next = top;
      top = element;
      Syso ("Element pushed");
      size++;
  }
  public int StackPop() {
      if (top == null) return -1;
      int topData = top.data;
      stackNode temp = top;
      top = top.next;
      return topData;
  }
```

```
public int StackSize() {
    return size;
}

public boolean StackIsEmpty() {
    return top == null;
}

public void printStack() {
    stackNode current = top;
    while ( current != null) {
        Syso (current.val);
        current = current.next;
    }
    Sysoln();
}
}

class Main() {
    public Static void main (String args()) {
        Stack s = new Stack();
        s.StackPush(10);
        s.StackPush (20);
        s.printStack();
        Sysoln ( s.StackPop());
        Sysoln (s.StackSize());
        Sysoln (s.StackIsEmpty());
    }
}
```

PAGE NO.
DATE: / /

Output:-

Element pushed
Element pushed
20 10
Element popped 20
Stack size: 2
Stack is empty or not: false

Time Complexity -

push() - O(1)
pop() - O(1)
peek() - O(1)
size() - O(1)

Space Complexity -

Required Space much better

Implement Stack Using 2 Queues -

① Element = 1 push

Queue2 - 1

queue2

क्या मेरा Queue1 खाली है? Yes -

Queue2 - 1

Queue1 - 1

② Element = 2

Queue2 = 2

क्या मेरा Queue1 खाली है? नही तो Queue1 के front से निकाल कर Queue2 में push करो

Queue1 = 1

Queue2 = 2 1

③ ~~Element = 3~~

Queue 2 = 2 1

क्या मेरा Queue खाली है? yes

Queue 2 = ~~2~~ ~~1~~

Queue 1 = 2 1

Element = 3

Queue 2 = 3

क्या मेरा Queue खाली है - नहीं ✗

Queue 2 = 3 2 1

Queue 1 = ~~2~~ ~~1~~

DRY RUN PUSH()

① Element = 1

push in queue2 = 1

queue1 खाली है क्या है ? → हाँ ✓

swap only — queue2 = 1

queue1 = 1

② Element = 2

push in queue2 = 2

queue1 खाली है क्या है ? → नहीं ✗

तो queue1 के elements queue2 में डालो

then

swap queue2 = 2 1

queue1 = 2 1

③ Element = 3

push in queue2 = 3

queue1 खाली है क्या → नहीं ✗

तो queue1 के elements queue2 में डालो

swap करो

queue2 = 3 2 1

queue1 = 3 2 1

④ Element = 4

queue2 = 4

queue1 खाली है क्या ? → नहीं ✗

queue1 के elements queue2 में डालो,
Then Swap —

queue2 = 4 3 2 1

queue1 = 4 3 2 1

⑤ Element = 5

queue2 = 5

queue1 खाली है क्या → नहीं ✗

queue1 के elements queue2 में डालो then swap.

queue2 = 5, 4, 3, 2, 1

queue1 = 5, 4, 3, 2, 1

// Code - (Using Two Queues) —

```
class MyStack {
    Queue<Integer> q1;
    Queue<Integer> q2;

    public MyStack() {
        q1 = new LinkedList<>();
        q2 = new LinkedList<>();
    }

    public void push(int x) {
        q2.offer(x);

        while(!q1.isEmpty()) {
            q2.offer(q1.poll());
        }

        // swap
        Queue<Integer> temp = q1;
        q1 = q2;
        q2 = temp;
    }

    public int pop() {
        int result = top();
        q1.poll();
        return result;
    }
```

```
public int top() {
    return q1.peek();
}

public boolean empty() {
    return q1.isEmpty();
}
}
```

- Time Complexity -

Top - O(1)

Pop - O(1)

Push - O(n)

Implement Stack Using 1 Queue –

① Element = 1

queue = 1

size = 1

loop ⇒ 0 ⟶ < size − 1 ⇒ < 0 ✗

② Element = 2

queue = 1 , 2

size = 2

loop ⇒ 0 ⟶ < size − 1 ⇒ 2 − 1 ⇒ < 1

0 → 0

queue = 2 , 1

③ Element = 3

queue = 2, 1, 3

size = 3

loop ⇒ 0 ⟶ size-1 ⇒ 3-1 ⇒ <2

0 ⟶ 1

queue = 3, 2, 1

④ Element = 4

queue = ~~4~~ 3, 2, 1, 4

size = 4

loop ⇒ 0 ⟶ size-1 ⇒ 4-1 ⇒ <3

0 ⟶ 2

queue = 4, 3, 2, 1

PAGE NO.
DATE: / /

Stack Using One Queue -

```
public class MyStack {
    private final Queue<Integer> q;

    public MyStack() {
        q = new LinkedList<>();
    }

    // Push element x onto Stack.
    public void push(int x) {                    O(n)
        q.offer(x);
        int rotations = q.size() - 1;
        for (int i = 0; i < rotations; i++) {
            q.offer(q.poll());
        }
    }

    // Removes the element on top of the Stack
    // and returns that element.
    public int pop() {                           O(1)
        if (q.isEmpty()) {
            System.out.println("Stack is empty");
        }
        return q.poll();
    }
```

DATE: / /

```
// Get the top Element
public int top() {                    O(1)
    if (q.isEmpty()) {
        Syso("Stack is empty");
    }
    return q.peek();
}

// Return whether the Stack is empty
public boolean empty() {
    return q.empty();                 O(1)
}

// Simple test
public static void main(String args[]) {
    MyStack Stack = new MyStack();
    Stack.push(1);
    Stack.push(2);
    Stack.push(3);
    Syso(Stack.top()); // 3
    Syso(Stack.pop()); // 3
    Syso(Stack.top()); // 2
    Stack.push(4);
    Syso(Stack.pop()); // 4
    while (!Stack.isEmpty()) {
        Syso(stack.pop());
    }
}
}
```

Space - O(1)

PAGE NO.
DATE: / /

Implementing Queue using Array :-

```
class Queue {
    private int arr[];
    private int start, end, currSize, maxSize;

    public Queue() {
        arr = new int[16];
        start = -1;
        end = -1;
        currSize = 0;
    }

    public Queue(int maxSize) {
        this.maxSize = maxSize;
        this.arr = new int[maxSize];
        start = -1;
        end = -1;
        currSize = 0;
    }

    public void push(int newElement) {
        if (currSize == maxSize) {
            Syso("Queue is full");
            System.exit(1);
        }

        if (end == -1) {
            start = 0;
            end = 0;
        }
```

DATE: / / /

```
else {
    end = (end + 1) % maxSize;
}
arr[end] = newElement;
Syso("The element pushed is" + newElement);
currSize++;
}

public int pop() {
    if (start == -1) {
        Syso("Queue is Empty");
        System.exit(1);
    }
    int popped = arr[start];
    if(currSize == 1) {
        start = -1;
        end = -1;
    }
    else {
        start = (start++) % maxSize;
    }
    currSize--;
    return popped;
}
```

```
public int peek() {
    if (start == -1) {
        Syso ("Queue is Empty");
        System.exit(1);
    }
    return arr[start];
}

public int size() {
    return cursize;
}
}

public class TUF {
    public static void main (String args[]) {
        Queue q = new Queue(6);
        q.push(4);
        q.push(14);
        q.push(24);
        q.push(34);

        Syso (q.peek());
        Syso (q.size());
        Syso (q.pop());
        Syso (q.peek());
        Syso (q.size());
    }
}
```

DATE: / /

Output :-

4
14
24
34
4
4
14
3

Time Complexity :-

pop function : O(1)
push function : O(1)
peek function : O(1)
size function : O(1)

Space Complexity :-

Whole Queue : O(n)

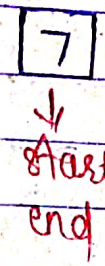
3b/

DRY RUN OF QUEUE using LinkedList

PAGE NO.
DATE: / /

Step 01:- start == end == -1 (null) size = 0

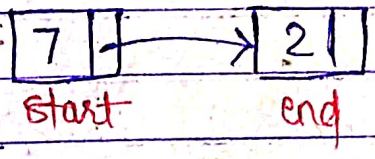
Step 02:- push(7) (size == 0) ← if start=end=null



size = 1

Step 03:- push(2) start ≠ end ≠ null

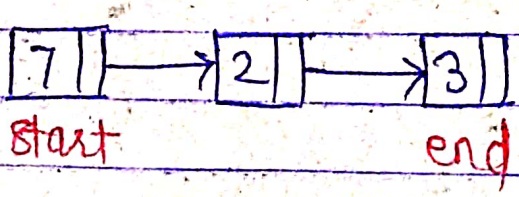
end → next ⇒ [2]



size = 2

Step 04:- push(3)

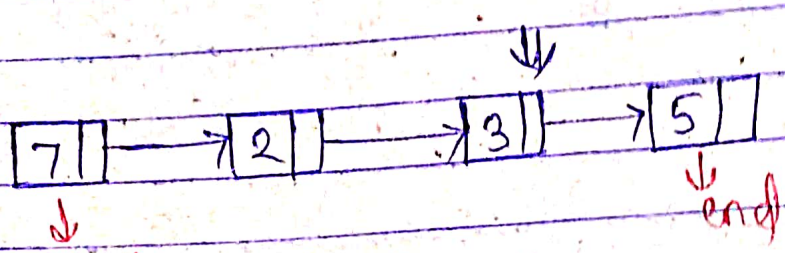
end.next → [3]



size = 3

Step 05:- push(5)

end.next → [5]



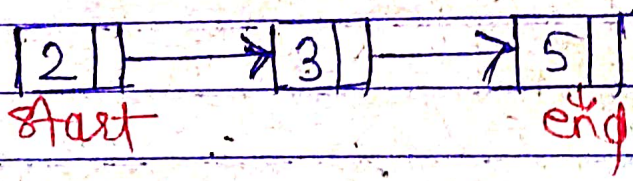
Size = 4

Step 06:- pop()

temp = start;

start = ~~start++~~; start.next;

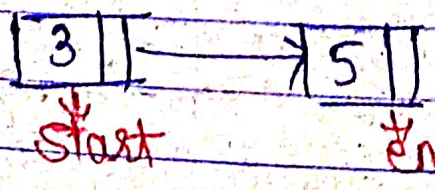
delete(temp); in C++



Step 07:- peek() = start.val;

Step 08:- pop()

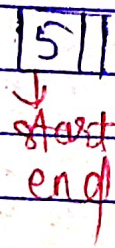
temp = start;
start = start.next;
delete(temp)



DATE:

Step 09 :- pop()

```
temp = start;
start = start.next;
delete(temp);
```



Step 10 :- peek() = start.val;

Step 11 :- pop()

```
temp = start;
start = start.next;
delete(temp);
```

if start == null then do end == null ...

Our Queue is Empty Now.

No Elements are left here...

PAGE NO.
DATE: / /

Implementing Queue Using LinkedList -

```
import java.util.*;

class QueueNode {
    int val;
    QueueNode next;
    QueueNode(int data) {
        val = data;
        next = null;
    }
}

class Queue {
    QueueNode front = null, Rear = null;
    int size = 0;

    boolean Empty() {
        return front == null;
    }

    int Peek() {
        if (Empty()) {
            Syso("Queue is Empty");
            return -1;
        }
        else {
            return front.val;
        }
    }
```

```
void Enqueue (int value) {
    QueueNode Temp;
    Temp = new QueueNode (value);

    if (Temp == null) {
        System.out.println ("Queue is full");
    }
    else {
        if (front == null) {
            front = Temp;
            Rear = Temp;
        }
        else {
            Rear.next = Temp;
            Rear = Temp;
        }
        System.out.println (value + " Inserted into Queue");
        size++;
    }
}
```

```
void Dequeue() {
    if (front == null) {
        Sysoln ("Queue is Empty");
    }
    else {
        Sysoln (front.val + "Removed from Queue");
        QueueNode Temp = front;
        front = front.next;
        size--;
    }
}

public static void main (String args[]) {
    Queue Q = new Queue();
    Q.Enqueue(10);
    Q.Enqueue(20);
    Q.Enqueue(30);
    Q.Enqueue(40);
    Q.Enqueue(50);
    Q.Dequeue();
    Sysoln ("The size of the Queue is" + Q.size);
    Sysoln (Q.peek());
}

}
```

Output :-

```
10 inserted into Queue
20 inserted into Queue
30 inserted into Queue
40 inserted into Queue
50 inserted into Queue
10 Removed from Queue
The size of the Queue is 4
The Peek element of the Queue is 20
```

Time Complexity :- O(1)

Space Complexity :- O(1)